

AI Assisted Coding

Name : B.Thanmai

Hall : 2303a52126

Batch : 33

Lab 3: Prompt Engineering – Improving Prompts and Context Management

Task Description-1

- Progressive Prompting for Calculator Design: Ask the AI to design a simple calculator program by initially providing only the function name. Gradually enhance the prompt by adding comments and usage examples.

The screenshot shows a VS Code editor with a Python file named `assignment3.2.py`. The code defines a `calculator` function that takes two numbers and an operator as input. It includes error handling for invalid operators and numeric inputs. The terminal shows the execution of the program, where the user enters the first number as 15, the second number as 5, and the operator as `+`. The result displayed is `15+5 = 20`.

```
10 def calculator(a=None, b=None, op=None):
11     print("Invalid operator. Choose one of: + - * /")
12
13     if a is None:
14         a = _get_number("Enter the first number: ")
15     if b is None:
16         b = _get_number("Enter the second number: ")
17     if op is None:
18         op = _get_operator("Enter an operator (+ - * /): ")
19
20     try:
21         a = float(a)
22         b = float(b)
23     except Exception:
24         print("Error: Invalid numeric input.")
25         return None
26
27     if op == '+':
28         result = a + b
29     elif op == '-':
30         result = a - b
31     elif op == '*':
32         result = a * b
33     elif op == '/':
34         result = a / b
35
36     print(f"{a} {op} {b} = {result}")
37     return result
38
39 def _get_number(prompt):
40     while True:
41         try:
42             value = input(prompt)
43             return float(value)
44         except ValueError:
45             print("Invalid input. Please enter a number.")
46
47 def _get_operator(prompt):
48     while True:
49         op = input(prompt)
50         if op in '+-*/':
51             return op
52         print("Invalid operator. Choose one of: + - * /")
53
54 if __name__ == '__main__':
55     calculator()
```

Warning: PowerShell detected that you might be using a screen reader and has disabled PSReadline for compatibility purposes. If you want to re-enable it, run 'Import-Module PSReadline'.

PS C:\Users\95425\Downloads\VAIAssistant> python -u "c:\Users\95425\Downloads\VAIAssistant\assignment3.2.py"

*Enter the first number: 15

Enter the second number: 5

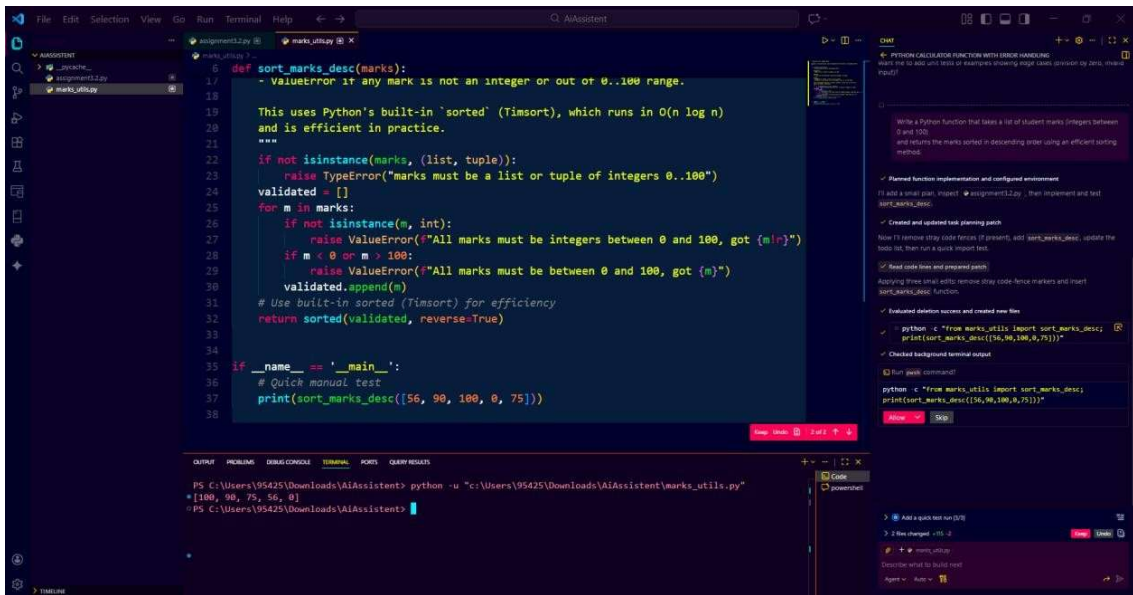
Enter an operator (+ - * /): +

Result: 15+5 = 20

PS C:\Users\95425\Downloads\VAIAssistant>

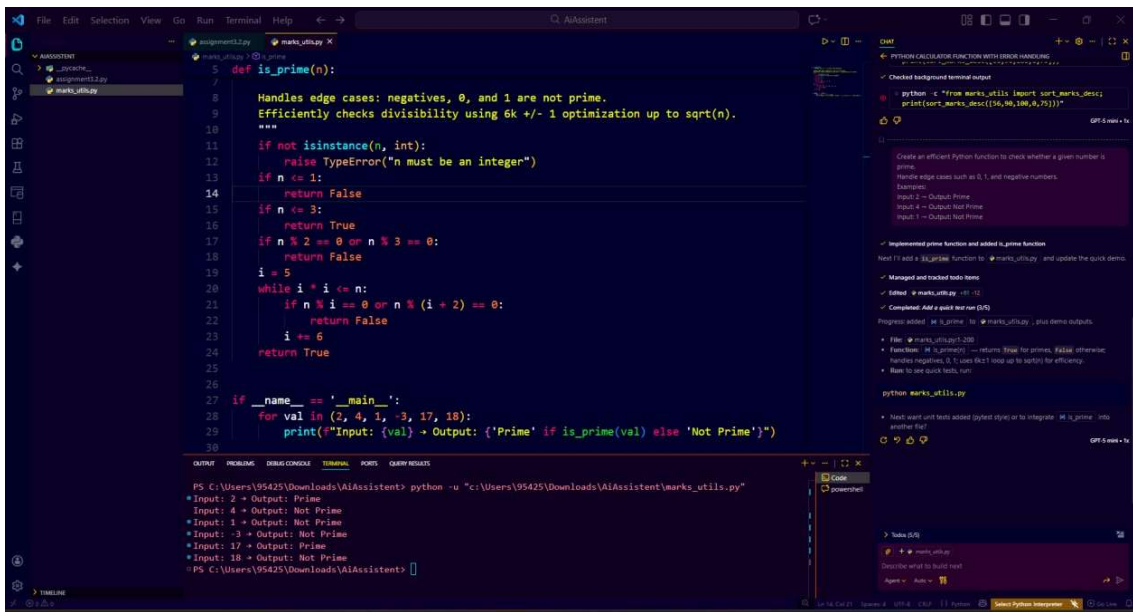
Task Description-2

- Refining Prompts for Sorting Logic: Start with a vague prompt for sorting student marks, then refine it to clearly specify sorting order and constraints.



Task Description-3

- Few-Shot Prompting for Prime Number Validation: Provide multiple input-output examples for a function that checks whether a number is prime. Observe how few-shot prompting improves correctness.



Task Description-4

- Prompt-Guided UI Design for Student Grading System: Create a user interface for a student grading system that calculates total marks, percentage, and grade based on user input.

```
File Edit Selection View Go Run Terminal Help
assignment2.py marks_utility.py grading_system.py

def grade_from_percentage(pct):
    """Return grade letter for percentage using given mapping.
    """
    A: >=90, B: >=75, C: >=60, D: <60
    """
    try:
        pct = float(pct)
    except Exception:
        return 'Invalid'
    if pct >= 90:
        return 'A'
    if pct >= 75:
        return 'B'
    if pct >= 60:
        return 'C'
    return 'D'

def grade_student(name, marks):
    """Compute total, percentage, and grade for a student.
    """
    'marks' may be a dict mapping subject->mark or an iterable of numeric marks.
    Returns a dict with 'name', 'marks' (as dict), 'total', 'percentage', 'grade'.
    Percentage is calculated as the average mark (treating each subject as out of 100).
    """
    # Normalize marks into an ordered dict-like mapping
    if isinstance(marks, dict):
        subj_marks = dict(marks)
    else:
        # assume iterable of marks without subject names
        subj_marks = {'Subject [i+1]': float(m) for i, m in enumerate(marks)}

    # Validate numeric marks and compute total
```

```
File Edit Selection View Go Run Terminal Help
assignment2.py marks_utility.py grading_system.py

def grade_student(name, marks):
    # Validate numeric marks and compute total
    total = 0.0
    count = 0
    for k, v in list(subj_marks.items()):
        try:
            mv = float(v)
        except Exception:
            mv = 0.0
        subj_marks[k] = mv
        total += mv
        count += 1

    if count == 0:
        percentage = 0.0
    else:
        average = total / count
        percentage = average # treating average as percentage (out of 100)

    grade = grade_from_percentage(percentage)
    return {
        'name': name,
        'marks': subj_marks,
        'total': total,
        'percentage': round(percentage, 2),
        'grade': grade,
    }

def interactive_grading_system():
    """Interactively accept multiple students and their subject marks.
    """
    For each student:
```

```
def interactive_grading_system():
    """Interactively accept multiple students and their subject marks.

    For each student:
    - Prompt for student name (blank to finish)
    - Prompt repeatedly for subject name and mark (blank subject to finish)
    - Validate numeric marks
    - Compute and display total, percentage, and grade in formatted output
    """
    results = []
    print('Enter student data. Leave student name blank to finish.')
    try:
        while True:
            name = input('\nStudent name: ').strip()
            if name == '':
                break

            subjects = {}
            print('Enter subject name and mark. Leave subject blank to finish.')
            while True:
                subj = input(' Subject name: ').strip()
                if subj == '':
                    break
                mark_raw = input(' Mark (0-100): ').strip()
                try:
                    mark = float(mark_raw)
                except ValueError:
                    print(' Invalid mark - please enter a numeric value.')
                    continue
                subjects[subj] = mark

            rec = grade_student(name, subjects)
            results.append(rec)

    except KeyboardInterrupt:
        print('\nInterrupted - returning to caller.')
    return results

if __name__ == '__main__':
    # Run the interactive grading system when executed directly.
    interactive_grading_system()
```

```
def interactive_grading_system():
    """Interactively accept multiple students and their subject marks.

    For each student:
    - Prompt for student name (blank to finish)
    - Prompt repeatedly for subject name and mark (blank subject to finish)
    - Validate numeric marks
    - Compute and display total, percentage, and grade in formatted output
    """
    results = []
    print('Enter student data. Leave student name blank to finish.')
    try:
        while True:
            name = input('\nStudent name: ').strip()
            if name == '':
                break

            subjects = {}
            print('Enter subject name and mark. Leave subject blank to finish.')
            while True:
                subj = input(' Subject name: ').strip()
                if subj == '':
                    break
                mark_raw = input(' Mark (0-100): ').strip()
                try:
                    mark = float(mark_raw)
                except ValueError:
                    print(' Invalid mark - please enter a numeric value.')
                    continue
                subjects[subj] = mark

            rec = grade_student(name, subjects)
            results.append(rec)

            # Display nicely
            print('\n---- Result ----')
            print(f'Student: {rec["name"]}')
            print(f'Subjects: {rec["subjects"]}')
            for s, m in rec["marks"].items():
                print(f' {s}: {m}')
            total_possible = len(rec["marks"]) * 100 if rec["marks"] else 0
            print(f'Total : {rec["total"]} / {total_possible}')
            print(f'Percent : {rec["percentage"]} %')
            print(f'Grade : {rec["grade"]}')
            print('-----')

            again = input('Add another student? (y/n): ').strip().lower()
            if again not in ('y', 'yes'):
                break

    except KeyboardInterrupt:
        print('\nInterrupted - returning to caller.')
    return results

if __name__ == '__main__':
    # Run the interactive grading system when executed directly.
    interactive_grading_system()
```

```
def interactive_grading_system():
    """Interactively accept multiple students and their subject marks.

    For each student:
    - Prompt for student name (blank to finish)
    - Prompt repeatedly for subject name and mark (blank subject to finish)
    - Validate numeric marks
    - Compute and display total, percentage, and grade in formatted output
    """
    results = []
    print('Enter student data. Leave student name blank to finish.')
    try:
        while True:
            name = input('\nStudent name: ').strip()
            if name == '':
                break

            subjects = {}
            print('Enter subject name and mark. Leave subject blank to finish.')
            while True:
                subj = input(' Subject name: ').strip()
                if subj == '':
                    break
                mark_raw = input(' Mark (0-100): ').strip()
                try:
                    mark = float(mark_raw)
                except ValueError:
                    print(' Invalid mark - please enter a numeric value.')
                    continue
                subjects[subj] = mark

            rec = grade_student(name, subjects)
            results.append(rec)

            # Display nicely
            print('\n---- Result ----')
            print(f'Student: {rec["name"]}')
            print(f'Subjects: {rec["subjects"]}')
            for s, m in rec["marks"].items():
                print(f' {s}: {m}')
            total_possible = len(rec["marks"]) * 100 if rec["marks"] else 0
            print(f'Total : {rec["total"]} / {total_possible}')
            print(f'Percent : {rec["percentage"]} %')
            print(f'Grade : {rec["grade"]}')
            print('-----')

            again = input('Add another student? (y/n): ').strip().lower()
            if again not in ('y', 'yes'):
                break

    except KeyboardInterrupt:
        print('\nInterrupted - returning to caller.')
    return results

if __name__ == '__main__':
    # Run the interactive grading system when executed directly.
    interactive_grading_system()
```


Task Description-5

- Analyzing Prompt Specificity in Unit Conversion Functions: Improving a Unit Conversion Function (Kilometers to Miles and Miles to Kilometers) Using Clear Instructions.

```
1 """Unit conversion tool
2
3 Defines 'convert_distance(value, unit)' and provides an interactive loop
4 to perform multiple conversions until the user exits.
5
6 Supported units: 'km' (kilometers) and 'miles'.
7 Conversion: 1 km = 0.621371 miles
8 """
9
10 def convert_distance(value, unit):
11     """Convert distance between kilometers and miles.
12
13     Parameters:
14     - value: numeric distance (int/float or string parseable to float)
15     - unit: string, either 'km' to convert km->miles or 'miles' to convert miles->km
16
17     Returns:
18     - converted_value (float rounded to 2 decimals), output_unit (str)
19
20     Raises:
21     - ValueError for invalid unit or non-numeric value
22     """
23     if unit is None:
24         raise ValueError('Unit must be specified as "km" or "miles"')
25     u = unit.strip().lower()
26     try:
27         v = float(value)
28     except Exception:
29         raise ValueError('Distance value must be numeric')
30
31     if u == 'km':
32         return (round(v * 0.621371, 2), 'miles')
33     if u == 'miles' or u == 'mile':
34         return (round(v / 0.621371, 2), 'km')
```

```
10 def convert_distance(value, unit):
11     u = unit.strip().lower()
12     try:
13         v = float(value)
14     except Exception:
15         raise ValueError('Distance value must be numeric')
16
17     if u == 'km':
18         return (round(v * 0.621371, 2), 'miles')
19     if u == 'miles' or u == 'mile':
20         return (round(v / 0.621371, 2), 'km')
21     raise ValueError(f'Invalid unit: {unit}. Use "km" or "miles"')
22
23 def interactive_convert_loop():
24     """Prompt user for conversions until they choose to exit."""
25     print('Unit converter - 1 km = 0.621371 miles')
26     try:
27         while True:
28             raw = input('Enter value and unit (e.g. '10 km' or '5 miles'), or 'q' to quit
29             if raw.lower() in ('q', 'quit', 'exit'):
30                 print('Goodbye.')
31                 break
32             if not raw:
33                 continue
34
35             parts = raw.split()
36             if len(parts) == 1:
37                 print('Please provide both value and unit, e.g. '10 km'.')
38                 continue
39             # join all but last as value in case of spaces in numbers (rare)
40             value_str = ' '.join(parts[:-1])
41             unit = parts[-1]
```

